



■ CS177 Bioinformatics: Software Development and Applications

Lecture 11: Generative AI

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2025



Learning objectives



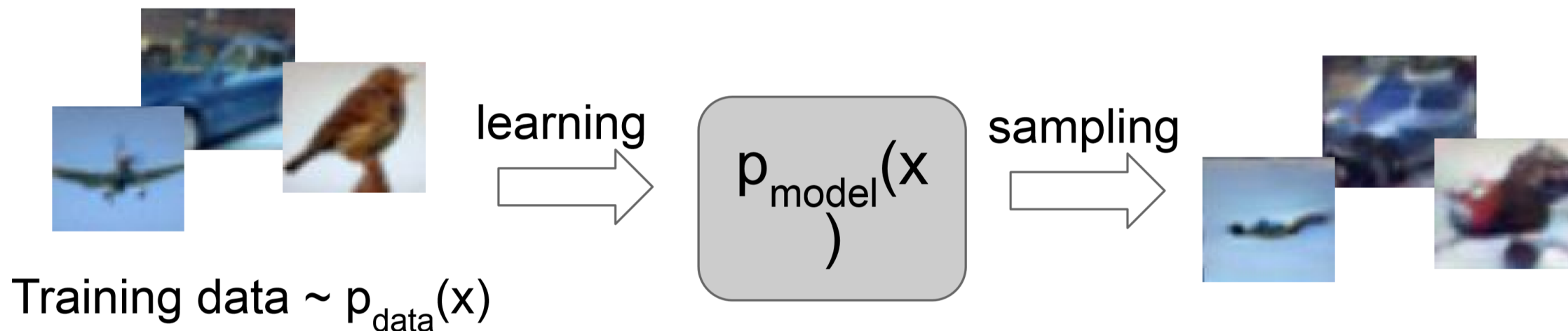
- At the end of this lecture, you should be able to:
 - Describe the basic rationale of generative modeling
 - Explain how Variational autoencoders (VAEs) work
 - Describe the basic idea behind Generative Adversarial Networks (GANs)
 - Explain how diffusion models works, e.g. forward diffusion and reverse denoising
 - Describe how to evaluate generative models



Generative modeling



- Given training data, generate new samples from the same distribution



Objectives:

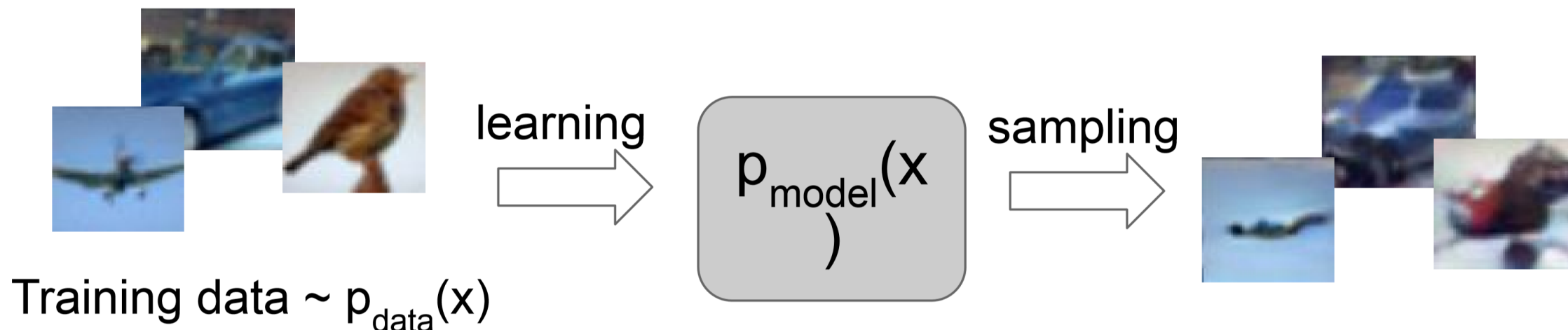
1. Learn $p_{\text{model}}(x)$ that approximates $p_{\text{data}}(x)$
2. Sample new x from $p_{\text{model}}(x)$



Generative modeling



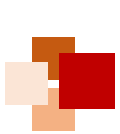
- Given training data, generate new samples from the same distribution



Formulate as **density estimation problems**:

- 1. Explicit density estimation**: explicitly define and solve for $p_{\text{model}}(x)$
- 2. Implicit density estimation**: learn model that can sample from $p_{\text{model}}(x)$ without explicitly defining it





Taxonomy of generative models

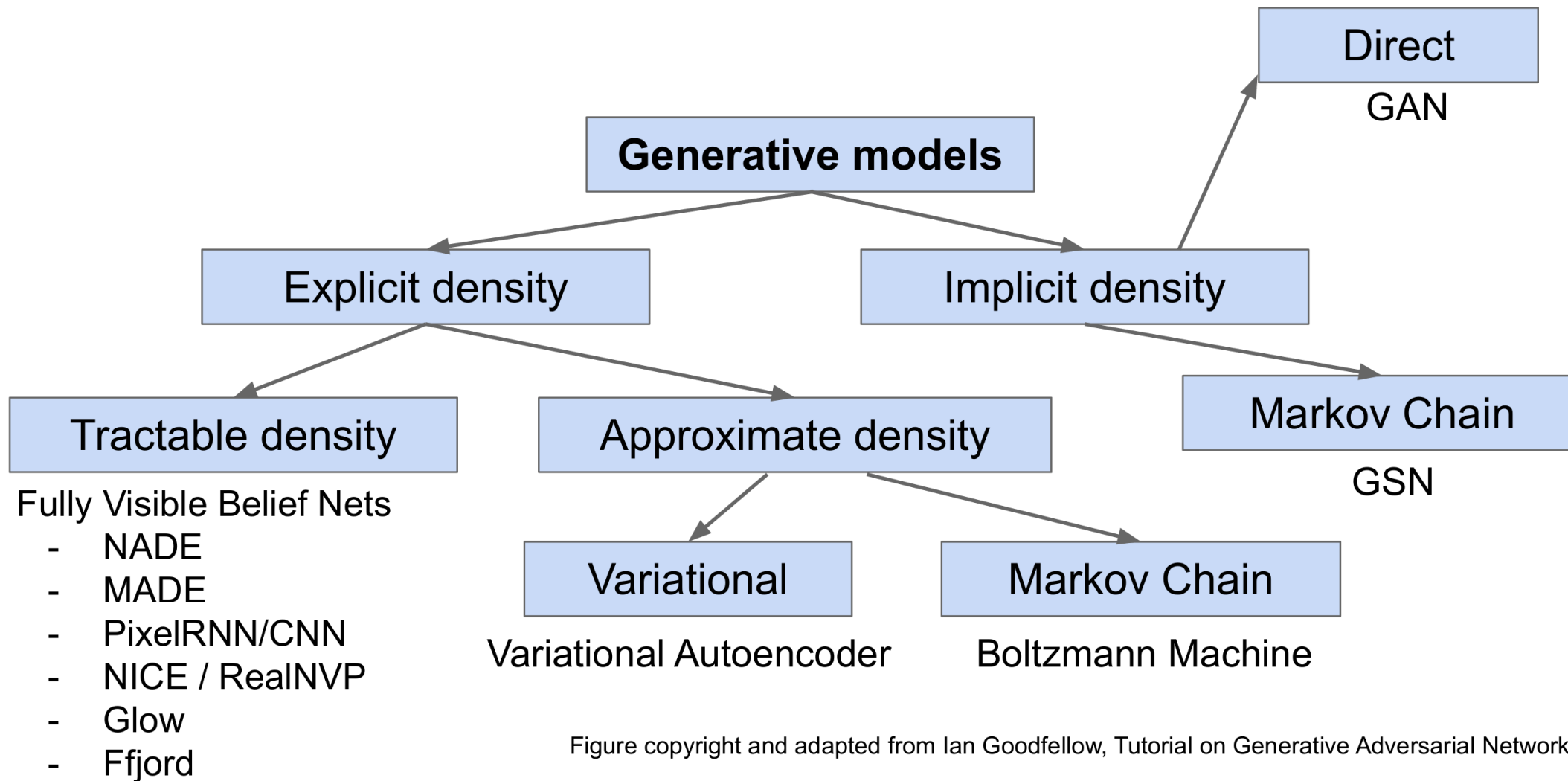
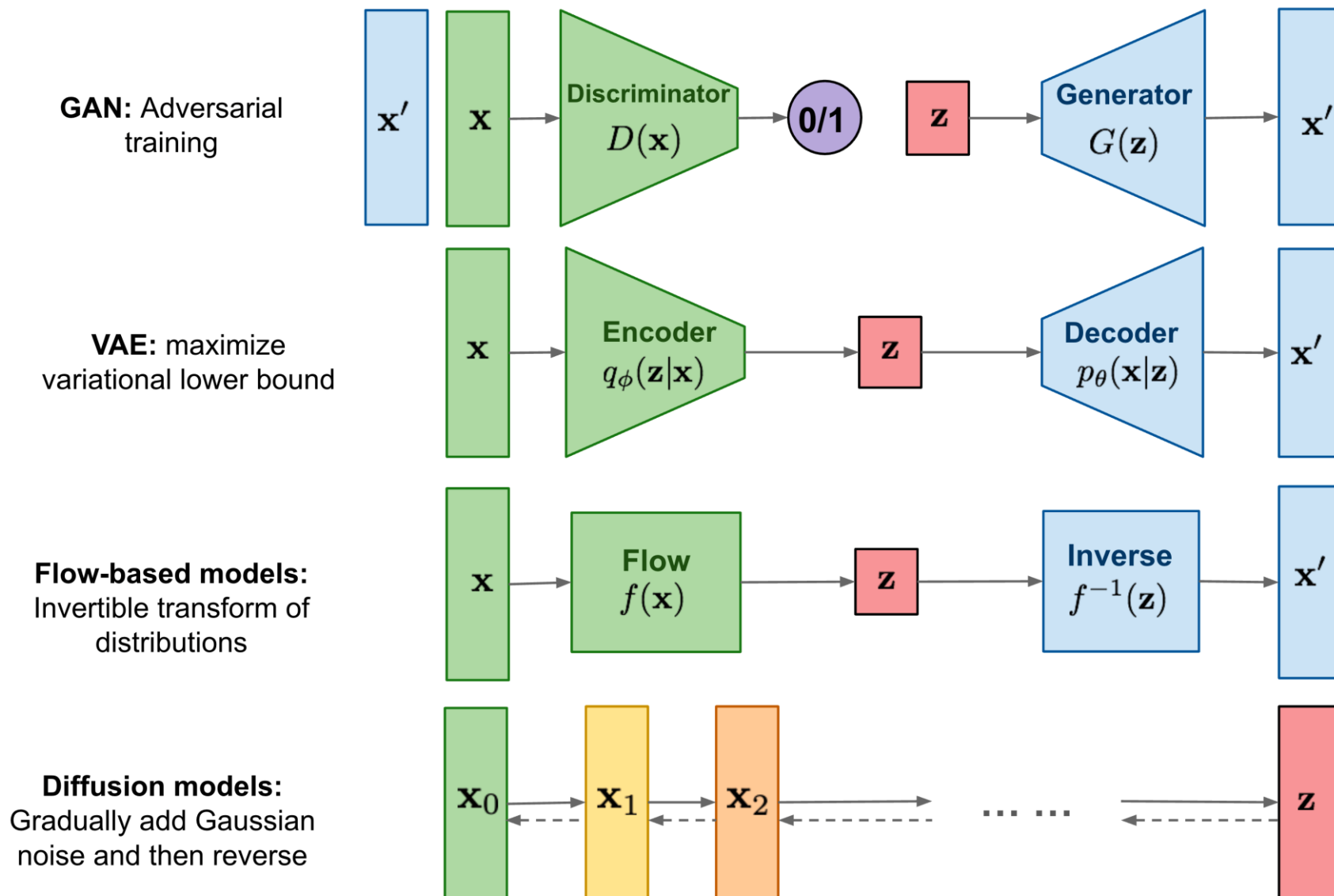


Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.



Generative models

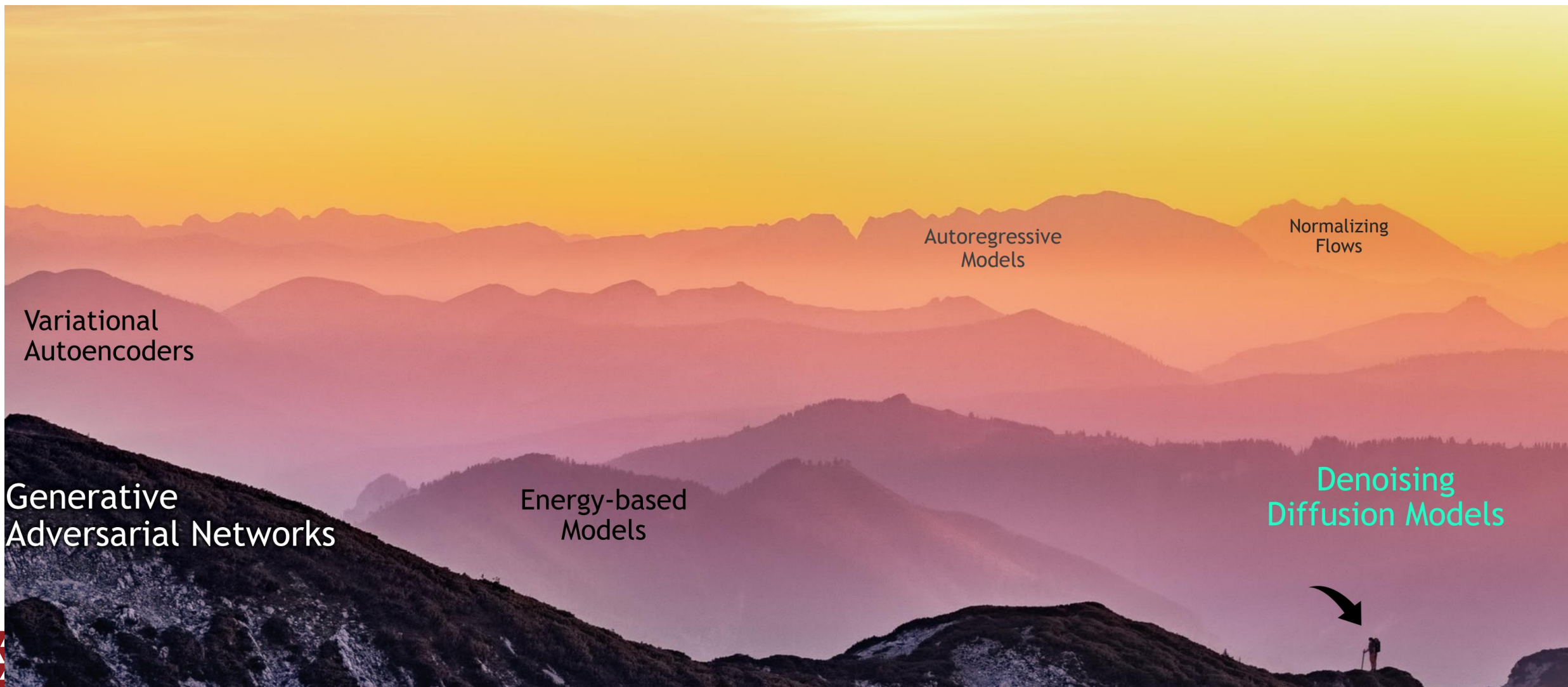




The landscape of deep generative learning



上海科技大学
ShanghaiTech University



立志成才 报國裕民



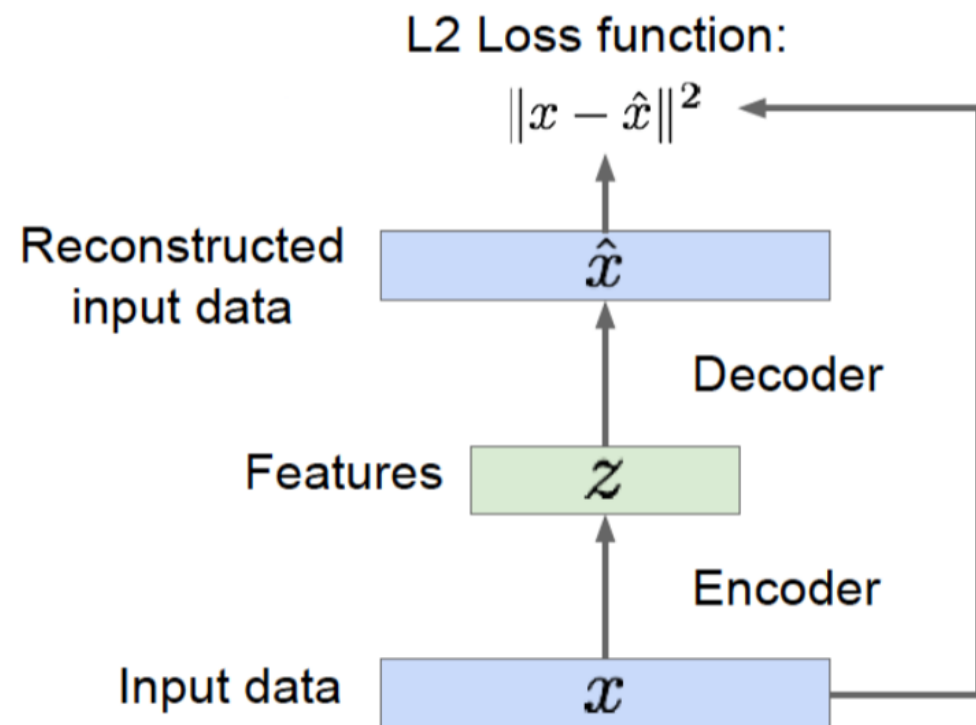
Variational Autoencoders (VAEs)

Some PPT slides from Stanford CS231n Lecture 11 (Prof. Fei-Fei Li)



Recap: Autoencoders

- By finding internal representations with lower dimensionality than the input data, we can find important patterns in the data
- **Autoencoder (AE)** is an **unsupervised** neural network for learning a lower-dimensional feature representation from unlabeled training data
 - It tries to reconstruct the input with lower dimensions (less neurons)





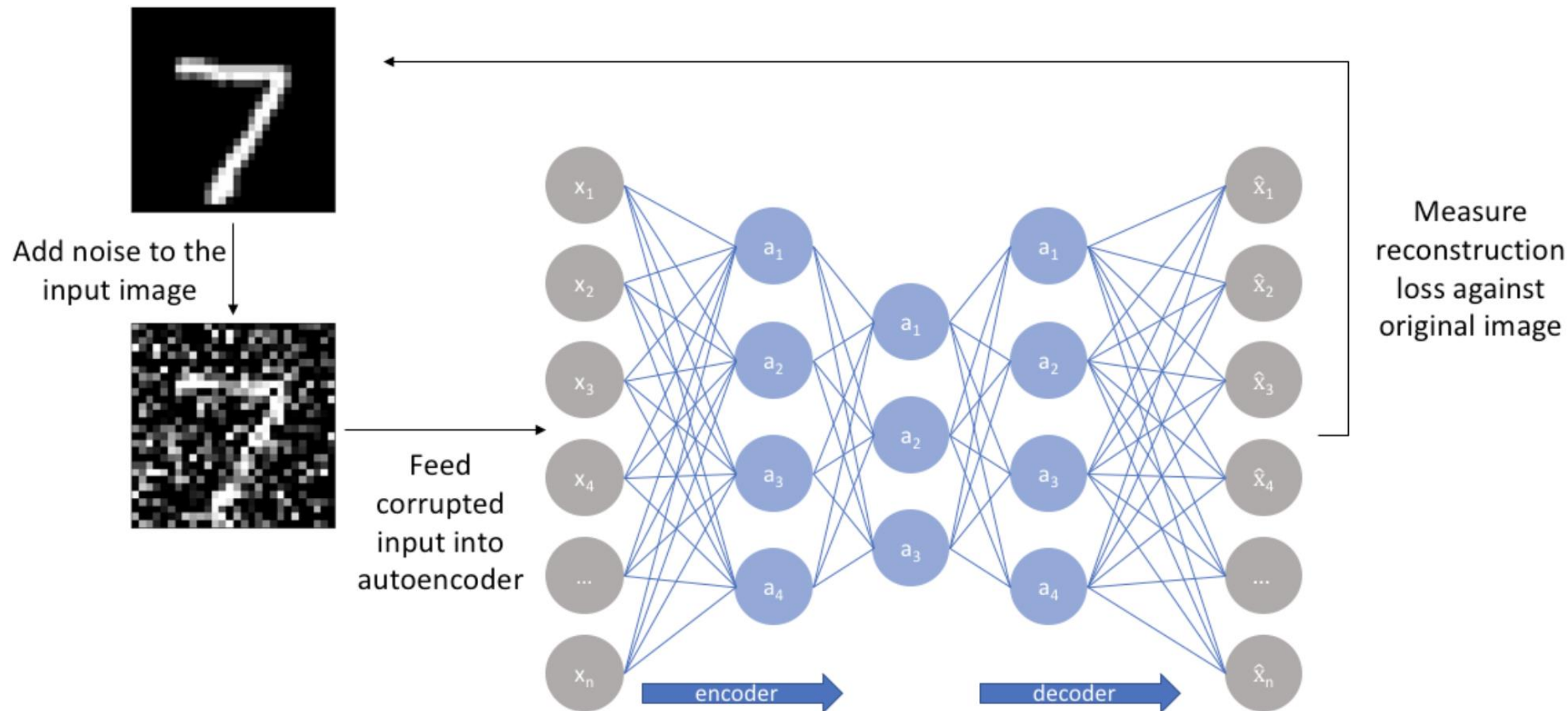
Recap: Denoising Autoencoder (DAE)

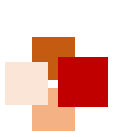


- What is DAE?
 - Randomly corrupt some of the inputs
 - Train the autoencoder to reconstruct the input from a corrupted version of it
 - The autoencoder is forced to predict the original inputs
 - This requires to capture the joint distribution between a set of variables
- **Rationale:** To increase the difficulty of reconstructing the output by adding noise

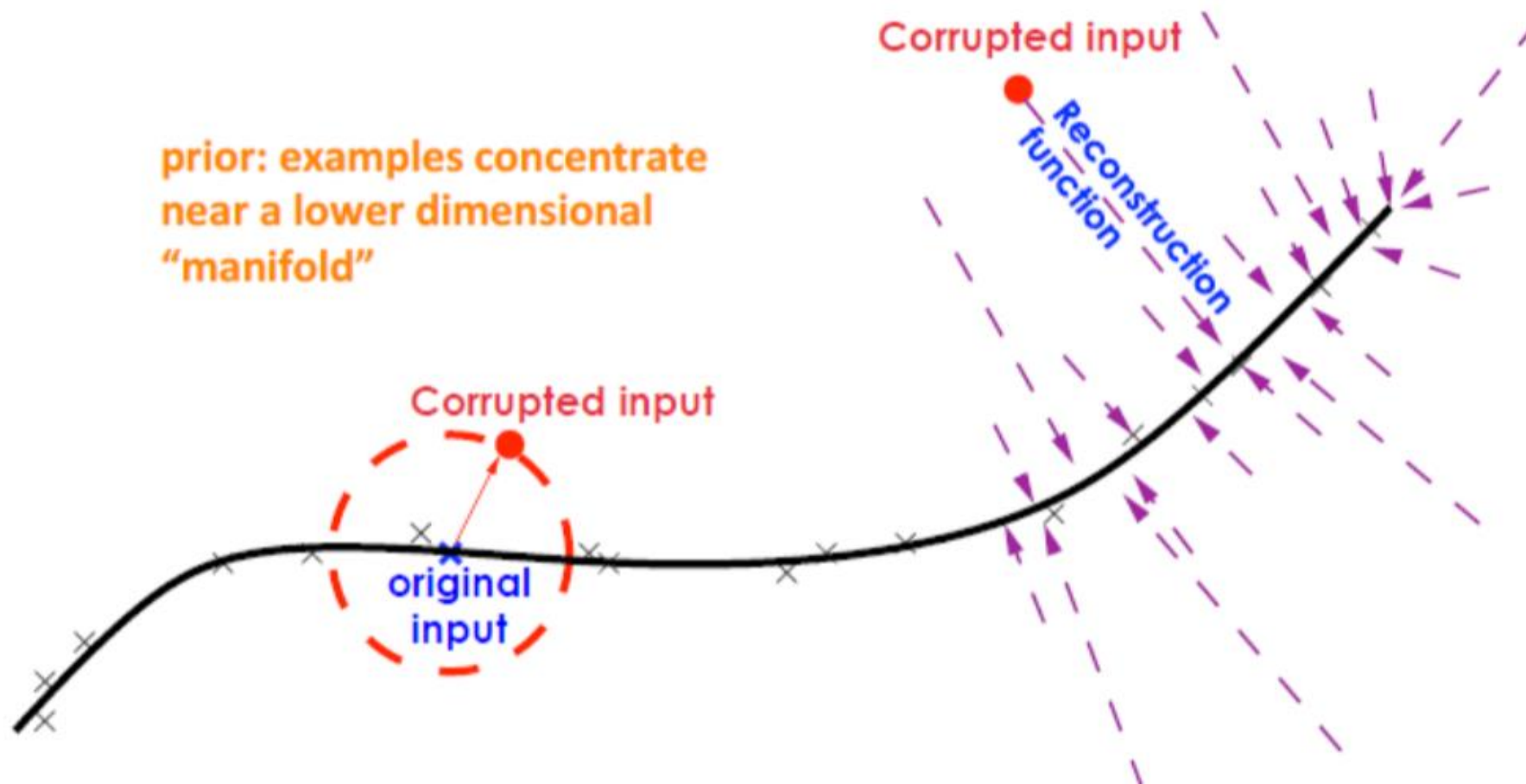


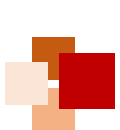
Recap: Structure of DAE



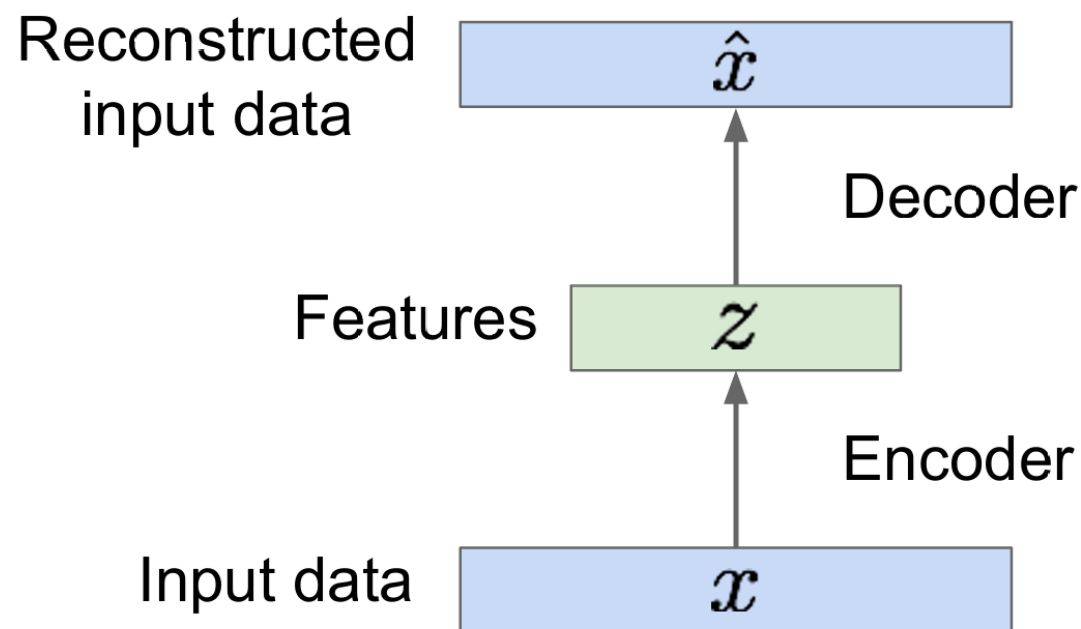


Recap: Geometric view of DAE





Make autoencoders generative models



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data.

But we can't generate new images from an autoencoder because we don't know the space of z .

How do we make autoencoder a **generative model**?





Variational Autoencoders (VAEs)

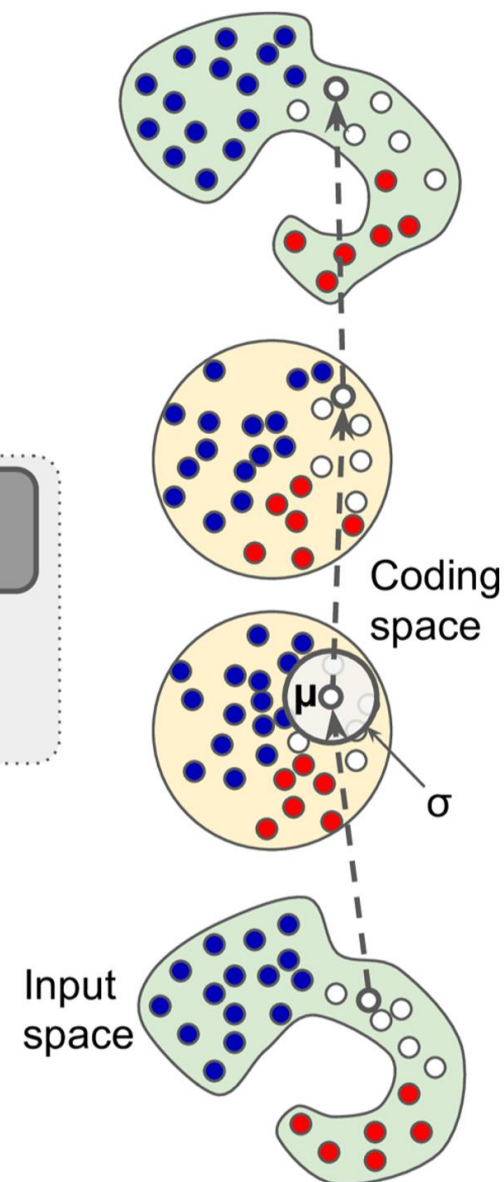
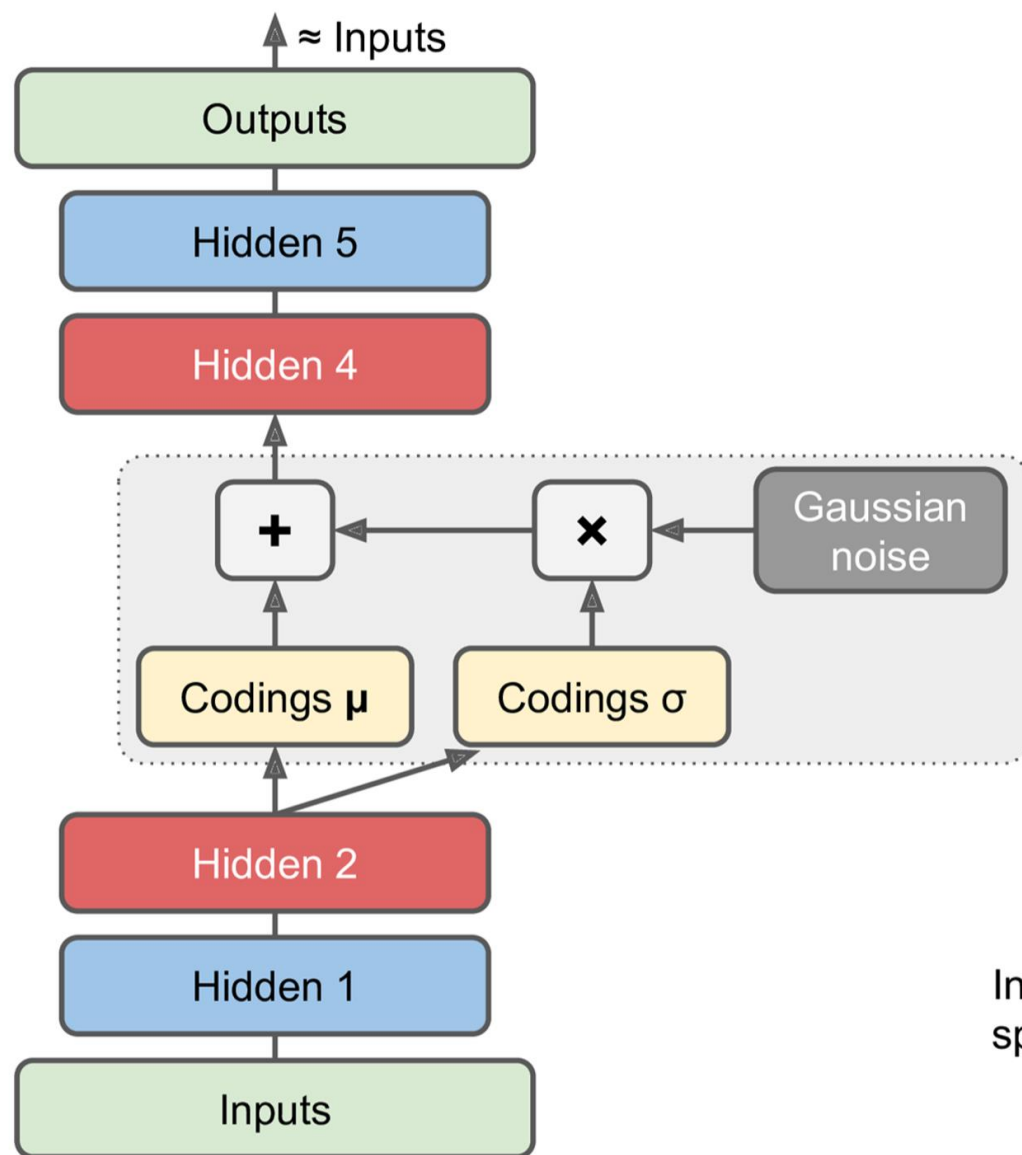


- A **variational autoencoder (VAE)** can be defined as being an autoencoder whose training is regularized
 - To avoid overfitting, and
 - To ensure that the latent space has good properties that enable generative process
- Instead of encoding an input as **a single point**, VAE encodes it as **a distribution** over the latent space





Variational Autoencoders (VAEs)





VAEs: How to represent the model

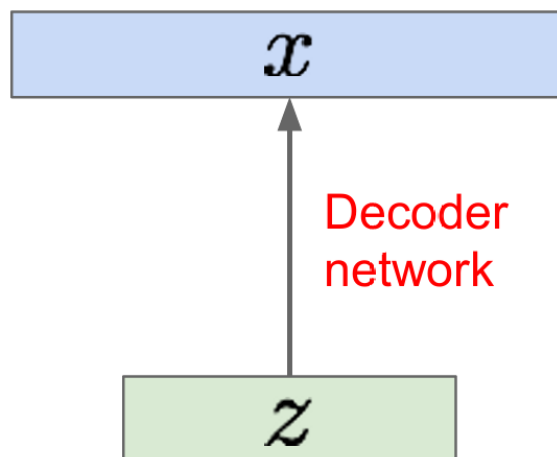


Sample from
true conditional

$$p_{\theta^*}(x | z^{(i)})$$

Sample from
true prior

$$z^{(i)} \sim p_{\theta^*}(z)$$



We want to estimate the true parameters θ^* of this generative model given training data x .

How should we represent this model?

Choose prior $p(z)$ to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g. pose, how much smile.

Conditional $p(x|z)$ is complex (generates image) => represent with neural network

Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014





VAEs: How to train the model



We want to estimate the true parameters θ^* of this generative model given training data x .

How to train the model?

Learn model parameters to maximize likelihood of training data

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Q: What is the problem with this?

Intractable!

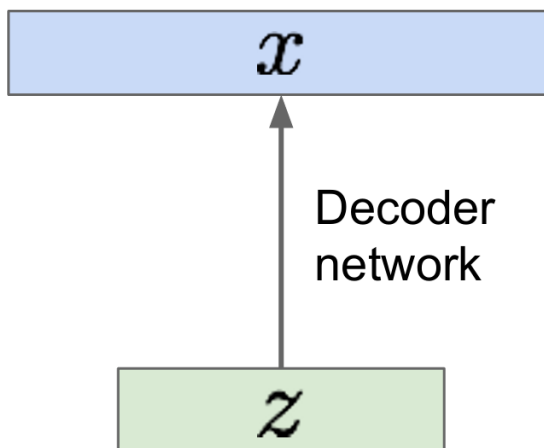
Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Sample from
true conditional

$$p_{\theta^*}(x | z^{(i)})$$

Sample from
true prior

$$z^{(i)} \sim p_{\theta^*}(z)$$



VAE: Tractable lower bound of likelihood



$$\log p_{\theta}(x^{(i)}) = \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} \left[\log p_{\theta}(x^{(i)}) \right] \quad (p_{\theta}(x^{(i)})) \text{ Does not depend on } z)$$

We want to
maximize the
data
likelihood

$$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)} | z) p_{\theta}(z)}{p_{\theta}(z | x^{(i)})} \right] \quad (\text{Bayes' Rule})$$

$$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)} | z) p_{\theta}(z)}{p_{\theta}(z | x^{(i)})} \frac{q_{\phi}(z | x^{(i)})}{q_{\phi}(z | x^{(i)})} \right] \quad (\text{Multiply by constant})$$

$$= \mathbf{E}_z \left[\log p_{\theta}(x^{(i)} | z) \right] - \mathbf{E}_z \left[\log \frac{q_{\phi}(z | x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[\log \frac{q_{\phi}(z | x^{(i)})}{p_{\theta}(z | x^{(i)})} \right] \quad (\text{Logarithms})$$

$$= \underbrace{\mathbf{E}_z \left[\log p_{\theta}(x^{(i)} | z) \right] - D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z | x^{(i)}))}_{\geq 0}$$

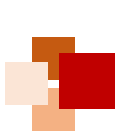
Also known as
**Evidence Lower
Bound (ELBO)**

Tractable lower bound which we can take
gradient of and optimize! ($p_{\theta}(x|z)$ differentiable,
KL term differentiable)



Generative Adversarial Networks (GANs)

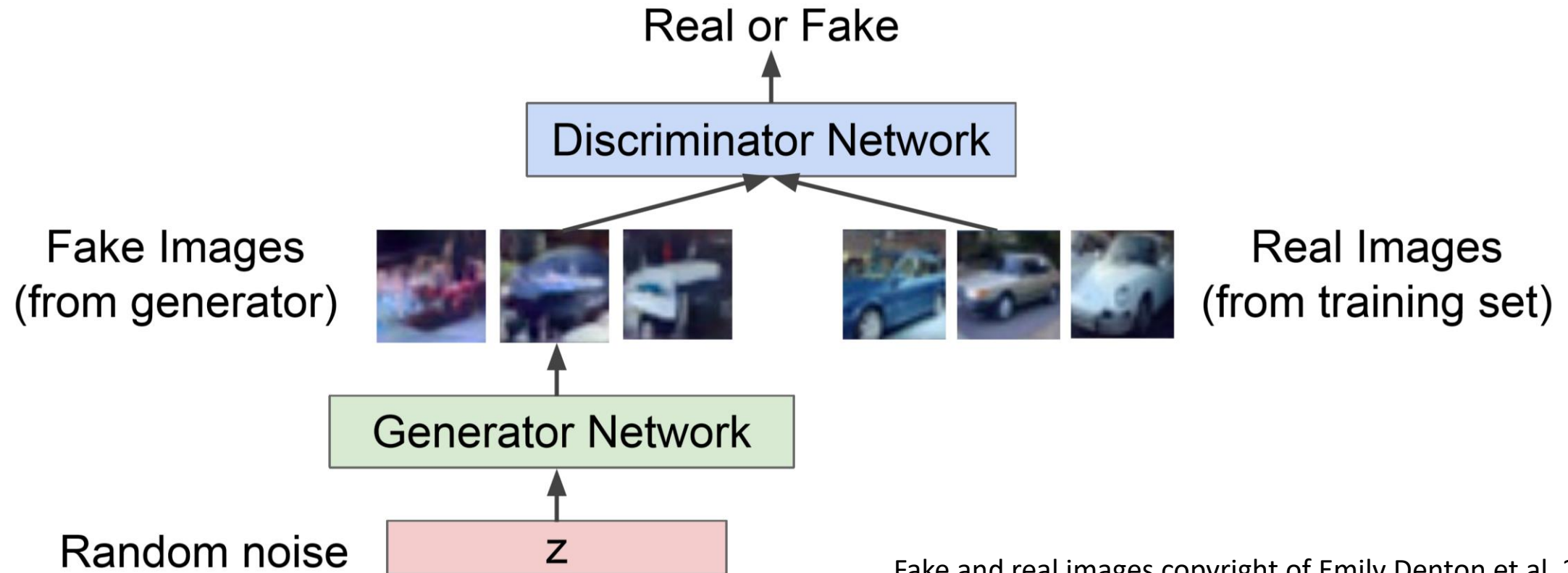




Generative Adversarial Networks (GANs)

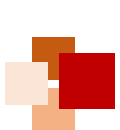


- Use a neural network, called **generator**, to generate data
- Use another neural network, called **discriminator**, to determine if the data is real or fake



Fake and real images copyright of Emily Denton et al. 2015.





Cost functions of adversarial learning



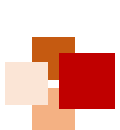
- Let D denote the discriminator's predicted probability of being real data
- **Discriminator:**
 - Try to distinguish between real and fake images
 - **Cost function:** cross-entropy loss for the task of classifying real vs. fake images:

$$\mathcal{J}_D = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[-\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[-\log(1 - D(G(\mathbf{z})))]$$

- **Generator:**
 - Try to fool the discriminator by generating real-looking images
 - **Cost function** (one possible version) is the opposite of the discriminator's:

$$\begin{aligned}\mathcal{J}_G &= -\mathcal{J}_D \\ &= \text{const} + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]\end{aligned}$$





Two-player game



- Minimax formulation:
 - The generator and discriminator are playing a **zero-sum game** against each other
 - Train jointly in a **minimax game**:

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

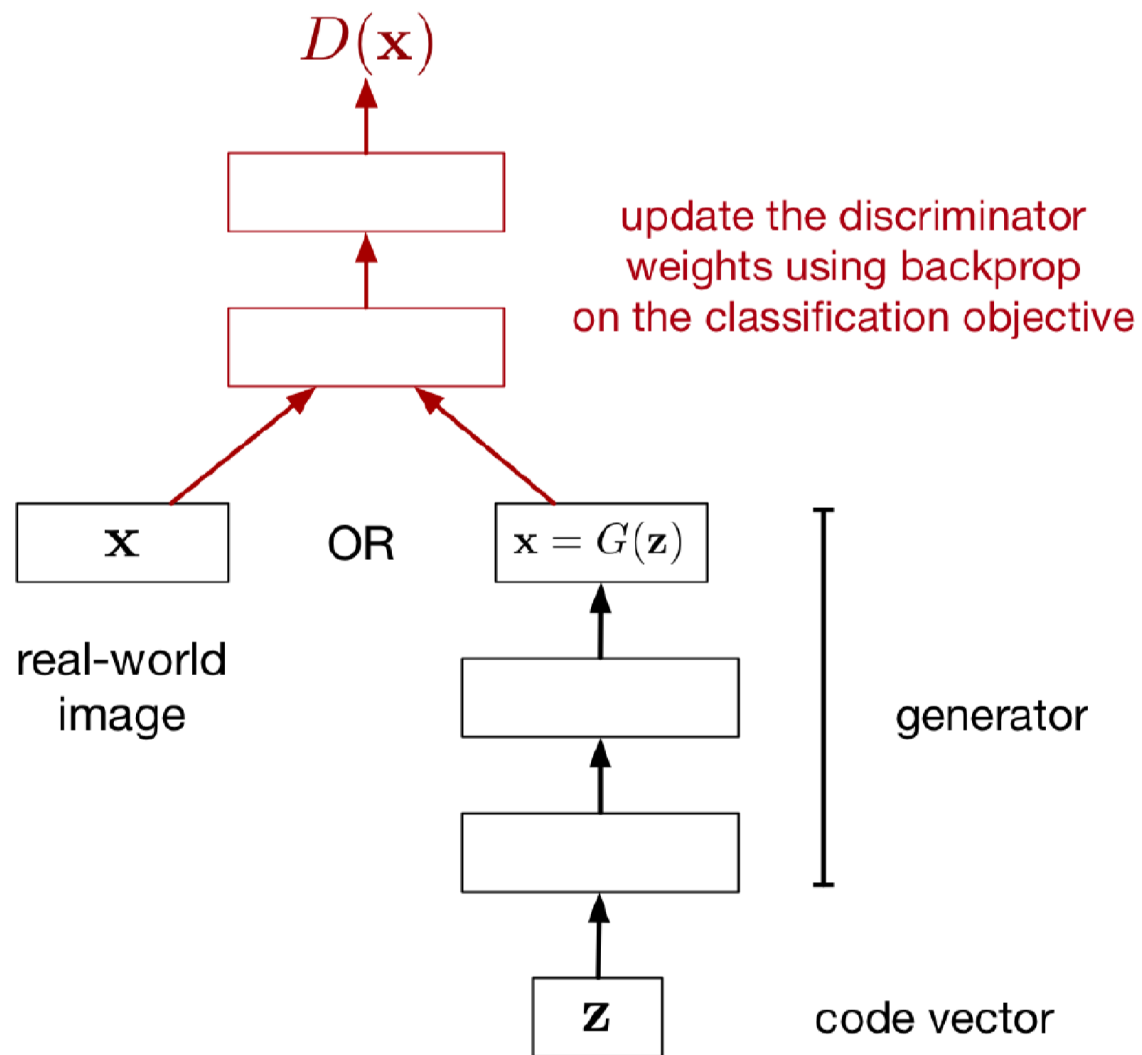
$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\substack{\text{Discriminator output} \\ \text{for real data } x}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\substack{\text{Discriminator output for} \\ \text{generated fake data } G(z)}}) \right]$$



Learning procedure



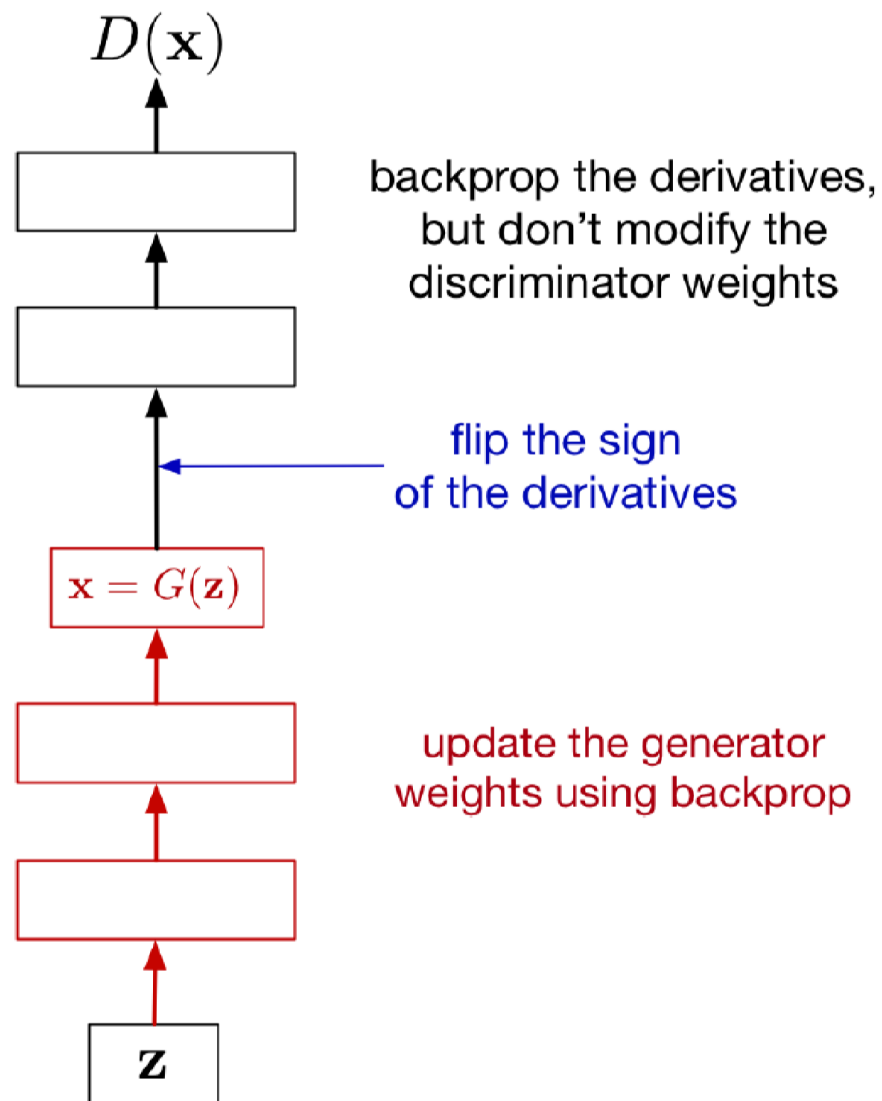
- Updating the discriminator



Learning procedure

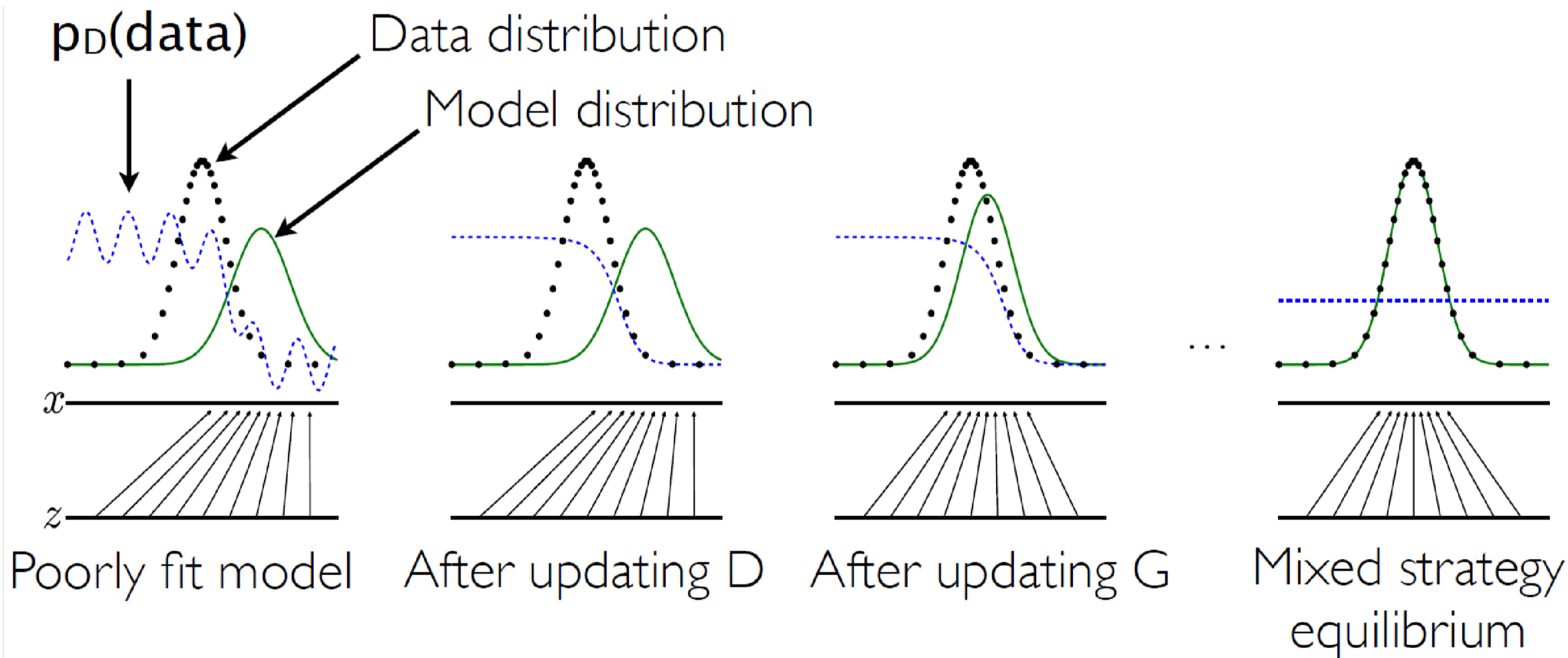


- Updating the generator





Training GANs



Modified from Fig. 1 of Ian J. Goodfellow et al. NIPS, 2014.





Training GANs



- In general, training a GAN is tricky and unstable
- Since GANs were introduced in 2014, there have been hundreds of papers introducing various architectures and training methods
- GAN Zoo: <https://github.com/hindupuravinash/the-gan-zoo>
- Many tricks:
 - S. Chintala, How to train a GAN, ICCV 2017 tutorial
 - <https://github.com/soumith/ganhacks>





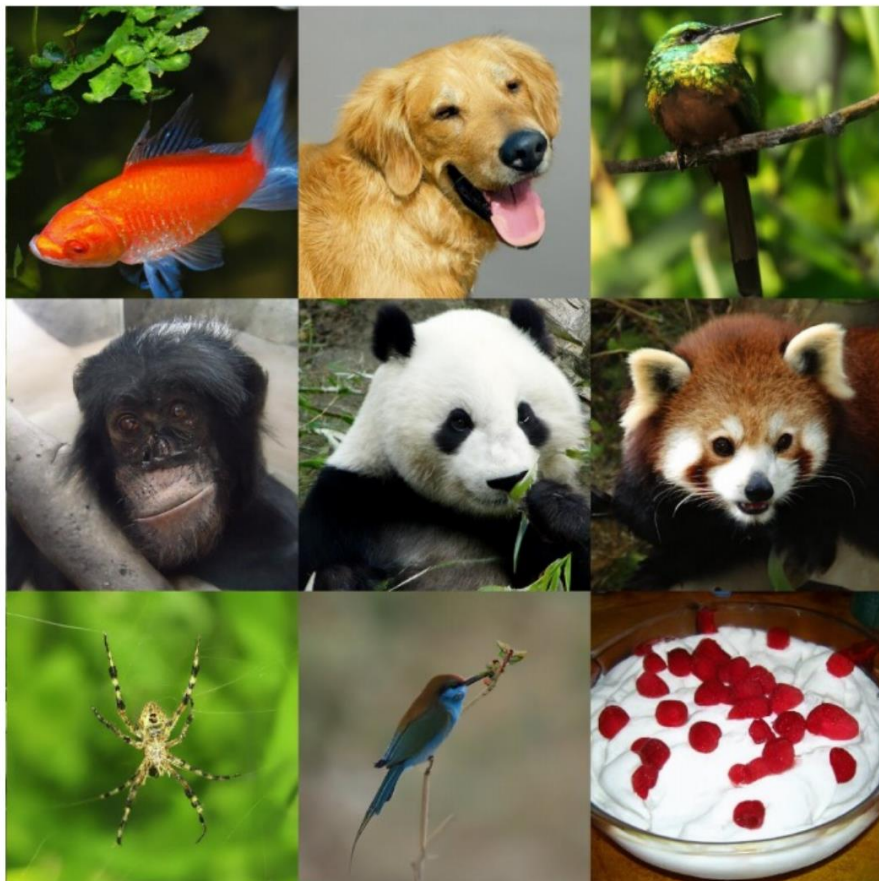
Diffusion Models

PPT mainly from CVPR 2022 tutorial “Denoising Diffusion-based Generative Modeling: Foundations and Applications” by K. Kreis, R. Gao and A. Vahdat



Denoising Diffusion Models

Emerging as powerful generative models, outperforming GANs



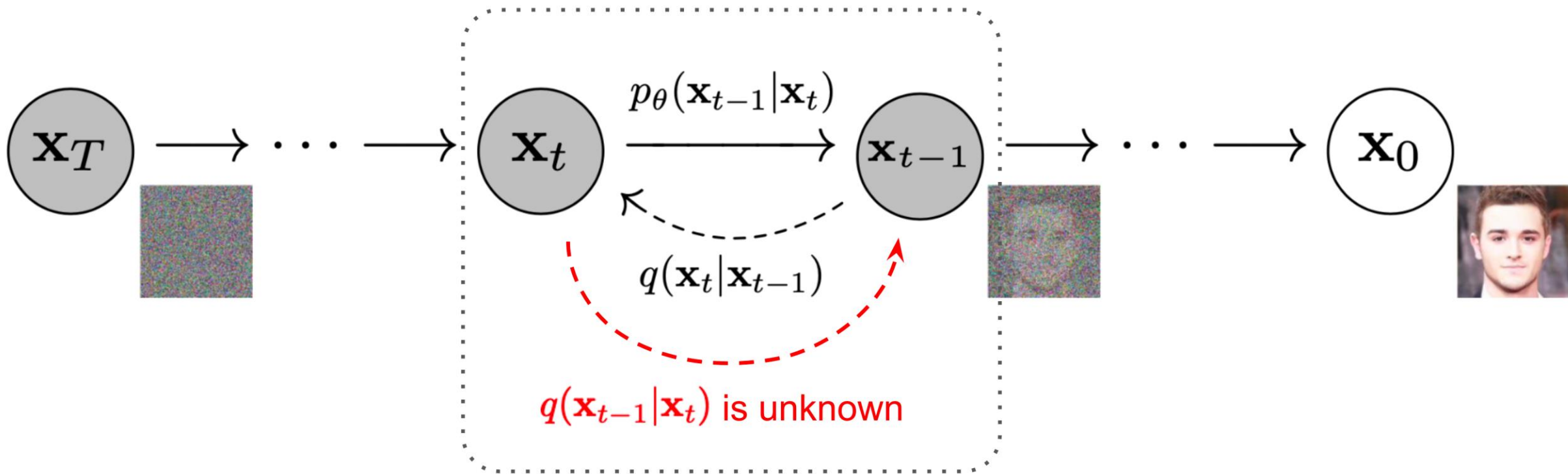
[“Diffusion Models Beat GANs on Image Synthesis”](#)
Dhariwal & Nichol, OpenAI, 2021



[“Cascaded Diffusion Models for High Fidelity Image Generation”](#)
Ho et al., Google, 2021

■ Denoising Diffusion Probabilistic Models (DDPMs)

Use variational lower bound



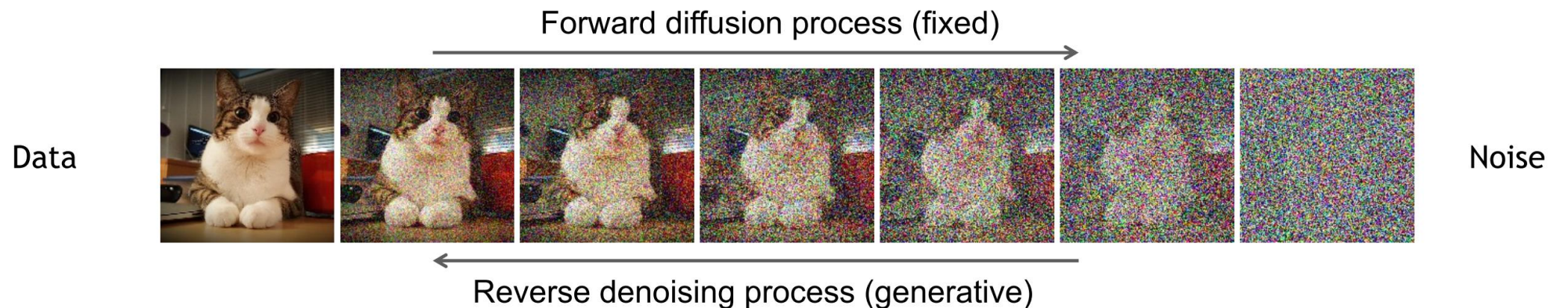
Ho, Jonathan, Ajay Jain, and Pieter Abbeel. NeurIPS 2020.

Denoising Diffusion Models

Learning to generate by denoising

Denoising diffusion models consist of two processes:

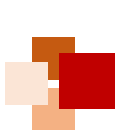
- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising



[Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015](#)

[Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020](#)

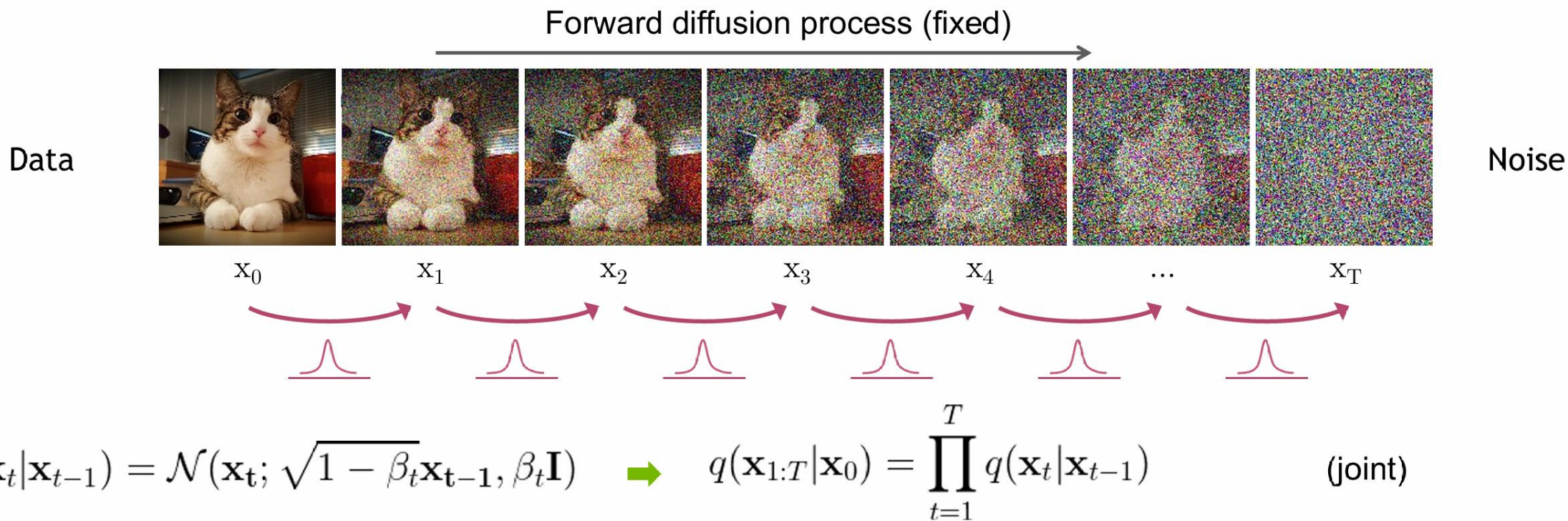
[Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021](#)

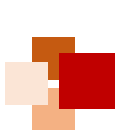


Forward diffusion process

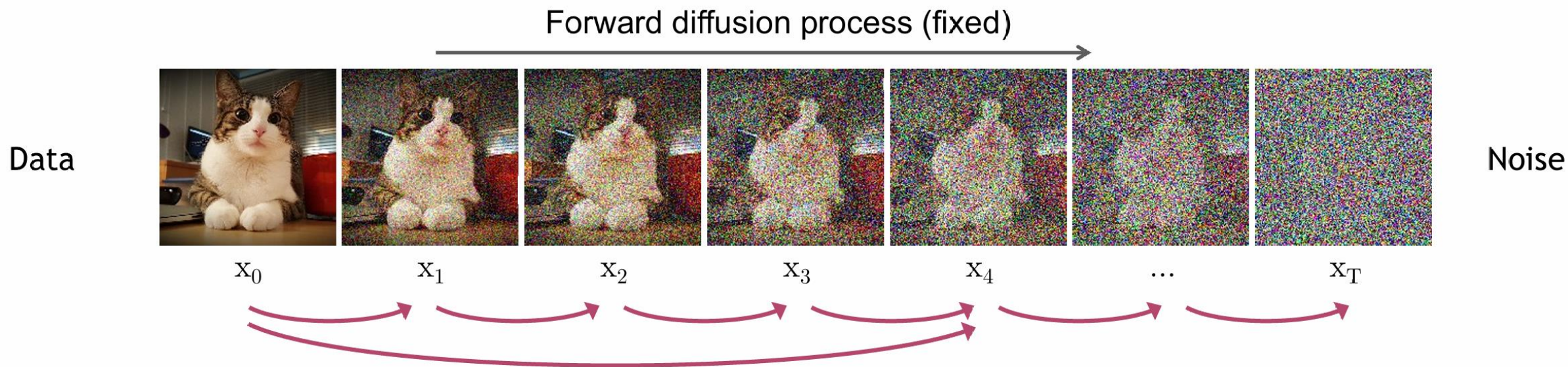


The formal definition of the forward process in T steps:





Diffusion kernel

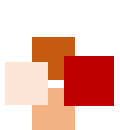


Define $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ ➡ $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$ (Diffusion Kernel)

For sampling: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

β_t values schedule (i.e., the noise schedule) is designed such that $\bar{\alpha}_T \rightarrow 0$ and $q(\mathbf{x}_T | \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$





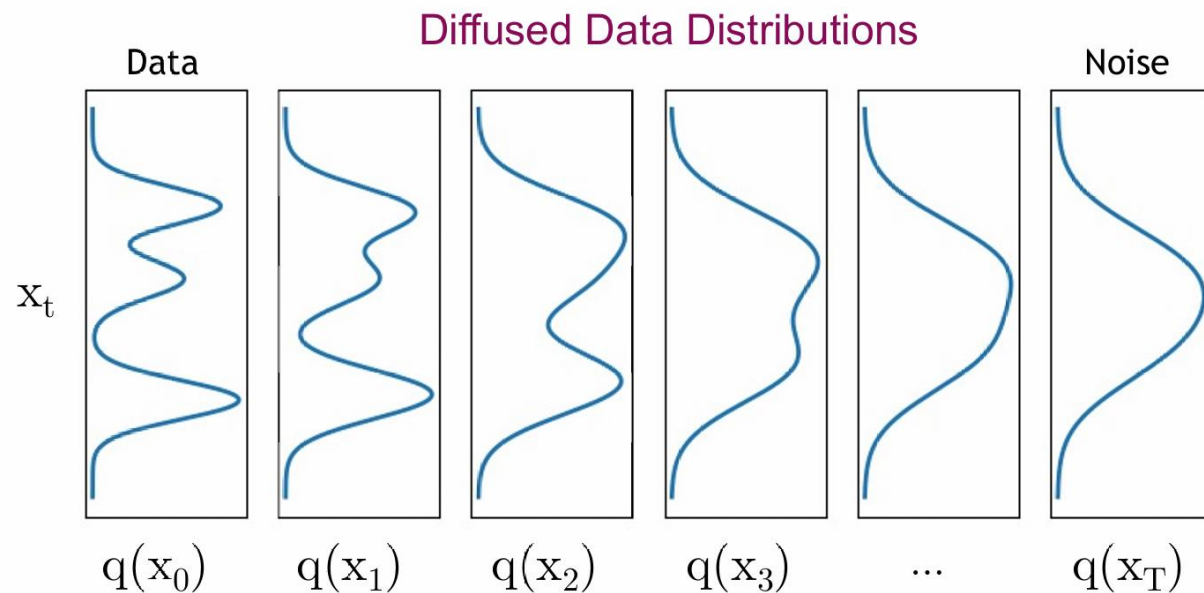
What happens to a distribution in the forward diffusion?



So far, we discussed the diffusion kernel $q(\mathbf{x}_t|\mathbf{x}_0)$ but what about $q(\mathbf{x}_t)$?

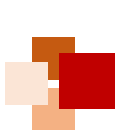
$$\underbrace{q(\mathbf{x}_t)}_{\text{Diffused data dist.}} = \int \underbrace{q(\mathbf{x}_0, \mathbf{x}_t)}_{\text{Joint dist.}} d\mathbf{x}_0 = \int \underbrace{q(\mathbf{x}_0)}_{\text{Input data dist.}} \underbrace{q(\mathbf{x}_t|\mathbf{x}_0)}_{\text{Diffusion kernel}} d\mathbf{x}_0$$

The diffusion kernel is Gaussian convolution.



We can sample $\mathbf{x}_t \sim q(\mathbf{x}_t)$ by first sampling $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ and then sampling $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$ (i.e., ancestral sampling).





Generative learning by denoising



Recall, that the diffusion parameters are designed such that $q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

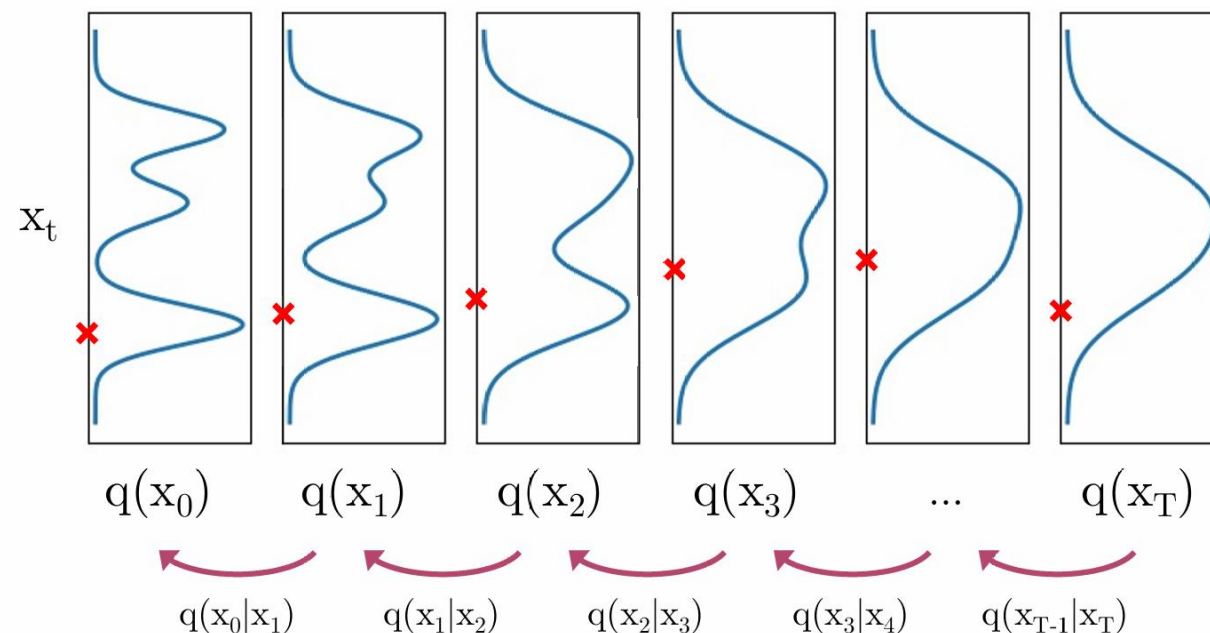
Generation:

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Iteratively sample $\mathbf{x}_{t-1} \sim \underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t)}$

True Denoising Dist.

Diffused Data Distributions



In general, $q(\mathbf{x}_{t-1}|\mathbf{x}_t) \propto q(\mathbf{x}_{t-1})q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is intractable.

Can we approximate $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$? Yes, we can use a **Normal distribution** if β_t is small in each forward diffusion step.

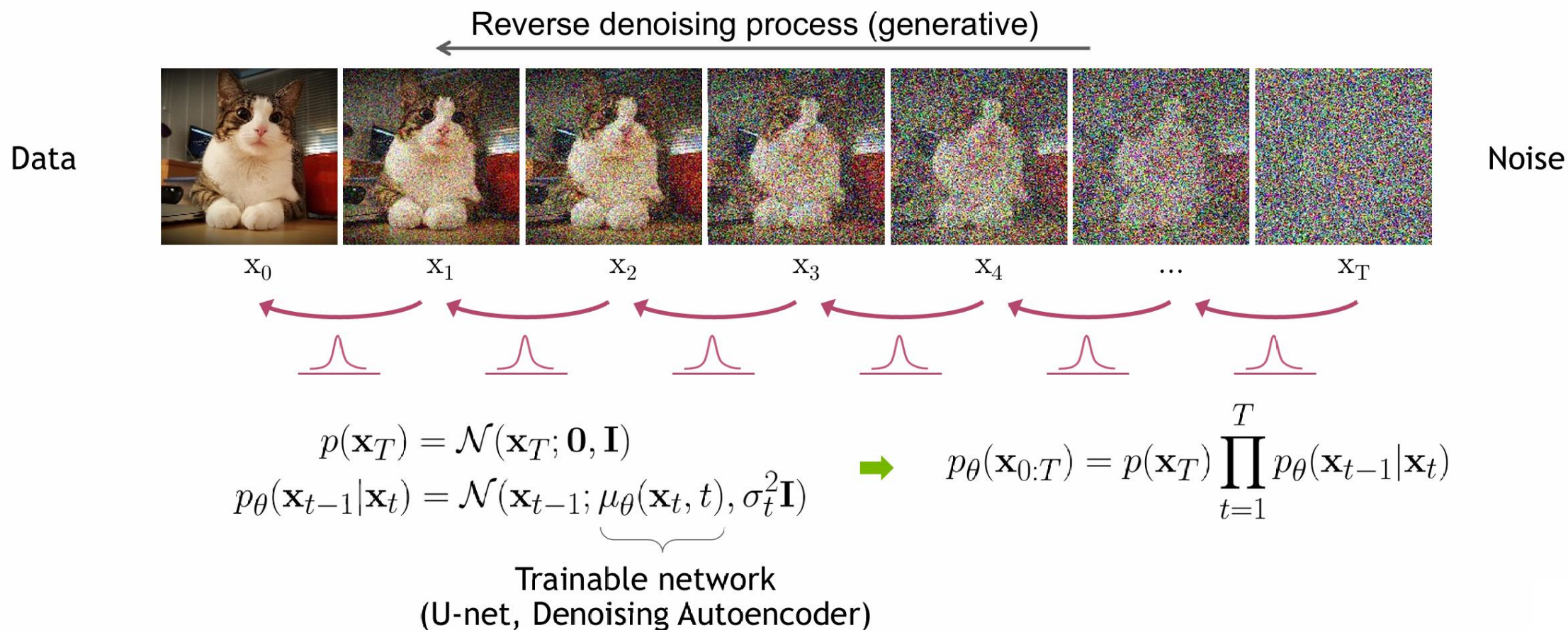




Reverse denoising process



Formal definition of forward and reverse processes in T steps:



Conditional diffusion models

Include condition as input to reverse process

Reverse process: $p_{\theta}(\mathbf{x}_{0:T}|\mathbf{c}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c}), \quad p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c}) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t, \mathbf{c}), \Sigma_{\theta}(\mathbf{x}_t, t, \mathbf{c}))$

Variational upper bound: $L_{\theta}(\mathbf{x}_0|\mathbf{c}) = \mathbb{E}_q \left[L_T(\mathbf{x}_0) + \sum_{t>1} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c})) - \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1, \mathbf{c}) \right].$

Incorporate conditions into U-Net

- Scalar conditioning: encode scalar as a vector embedding, simple spatial addition or adaptive group normalization layers.
- Image conditioning: channel-wise concatenation of the conditional image.
- Text conditioning: single vector embedding - spatial addition or adaptive group norm / a seq of vector embeddings - cross-attention.

Impressive conditional diffusion models: text-to-image generation



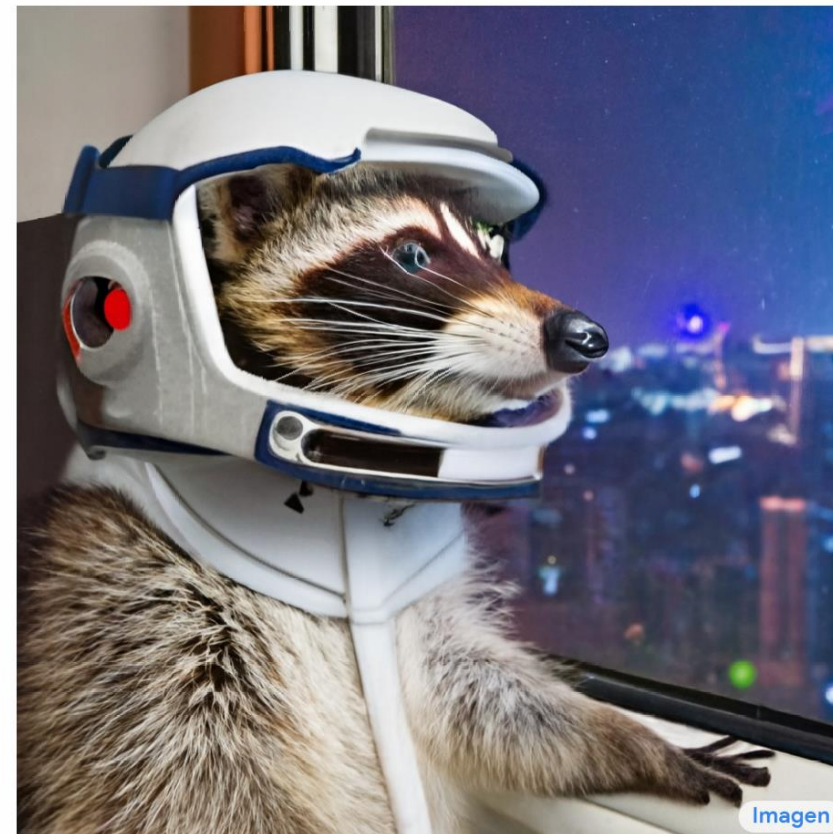
DALL·E 2

“a propaganda poster depicting a cat dressed as french emperor napoleon holding a piece of cheese”



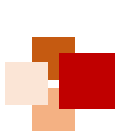
IMAGEN

“A photo of a raccoon wearing an astronaut helmet, looking out of the window at night.”

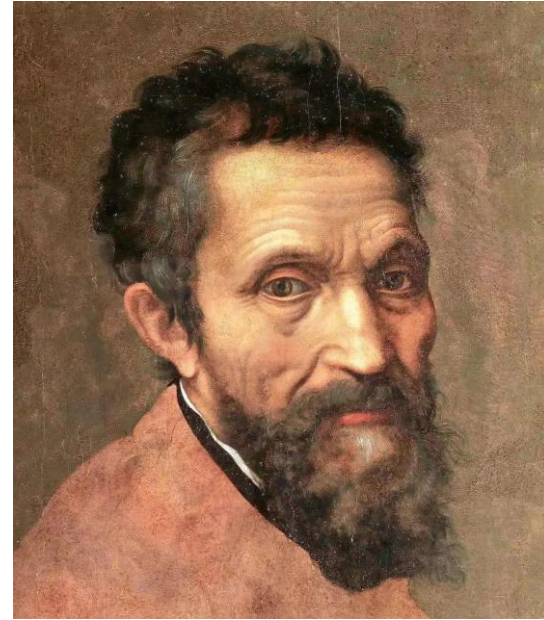


[Ramesh et al., “Hierarchical Text-Conditional Image Generation with CLIP Latents”, arXiv 2022.](#)

[Saharia et al., “Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding”, arXiv 2022.](#)



Denoising in the art creation



When they asked Michelangelo how he made his statue of David, he is reported to have said: "It is easy. You just chip away the stone that doesn't look like David. " — **Michelangelo**



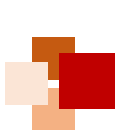


Evaluation of generative models



- In any research field, evaluation drives progress. How do we evaluate generative models?
- Evaluation of *discriminative* models (e.g., a classifier) is well understood: compare task-specific loss (e.g., top-1 accuracy) on unseen test data
- Evaluating generative models is highly non-trivial.
- **Key question:** What is the task that you care about?
 - Density estimation
 - Compression
 - Sampling/generation
 - Latent representation learning
 - More than one task? Custom downstream task? E.g., Semisupervised learning, image translation, compressive sensing etc. For LLMs: Few shot / zero shot performance through prompting?





Evaluation: Sample quality



vs.



$$S_1 = \{\mathbf{x} \sim P\}$$

$$S_2 = \{\mathbf{x} \sim Q\}$$

- Which of these two sets of generated samples “look” better?
- Human evaluations (e.g., Mechanical Turk) are the gold standard.

HYPE: Human eYe Perceptual Evaluation (Zhou et al., 2019)

- HYPE_{time}: the minimum time people needed to make accurate classifications. The larger, the better.
- HYPE_∞: The percentage of samples that deceive people under unlimited time. The larger, the better.
- <https://stanfordhci.github.io/gen-eval/>





References



- Chapter 17 of Aurélien Géron' s book “Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow” (Ed. 2) , O’ Reilly, 2019. Chapter on VAE and GAN
- Ziqiao Zhang et al. Shuigeng Zhou (Fudan University), “GANs for Molecule Generation in Drug Design and Discovery” , Chapter 11 of Book “Generative Adversarial Learning: Architectures and Applications” , Springer Nature, 2022.
- Jonathan Ho et al. “Denoising Diffusion Probabilistic Models” , NeurIPS 2020.
- Calvin Luo et al. “Understanding Diffusion Models: A Unified Perspective” , arXiv 2022.
- <https://cvpr2022-tutorial-diffusion-models.github.io/> (CVPR 2022 Tutorial on Denoising diffusion-based generative modeling)





Appendix:

An example of GAN in drug design: MolGAN





Drug discovery and design



- **Goal:** To find molecules with desirable properties, including bioactivity, physico-chemical properties, ADMET properties, etc.
- Drug development process consists of steps, such as:
 - Hit discovery
 - Lead optimization
 - Pre-clinical and clinical trials, etc.

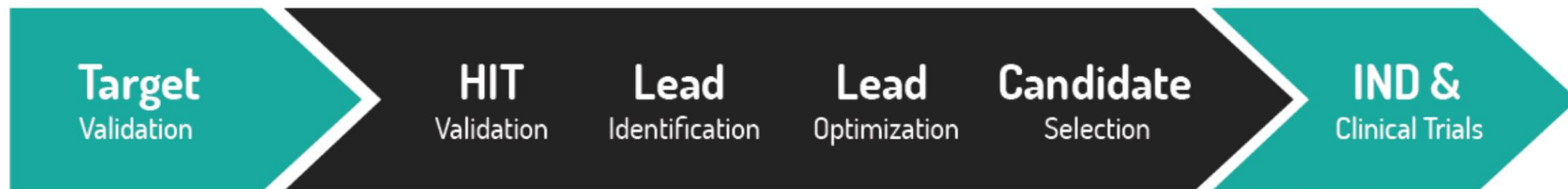


Figure from: <https://naeja-rgm.com/Drug-Discovery/About>





Drug discovery and design (continued)



- The process before clinical trials is a stack of **iterative** procedures with two phases:
 - Structure discovery:** Researchers navigate in the molecular space to find molecular structures that meet the requirements of concerned properties
 - Test and analysis:** The concerned properties of these molecules are tested and analyzed.

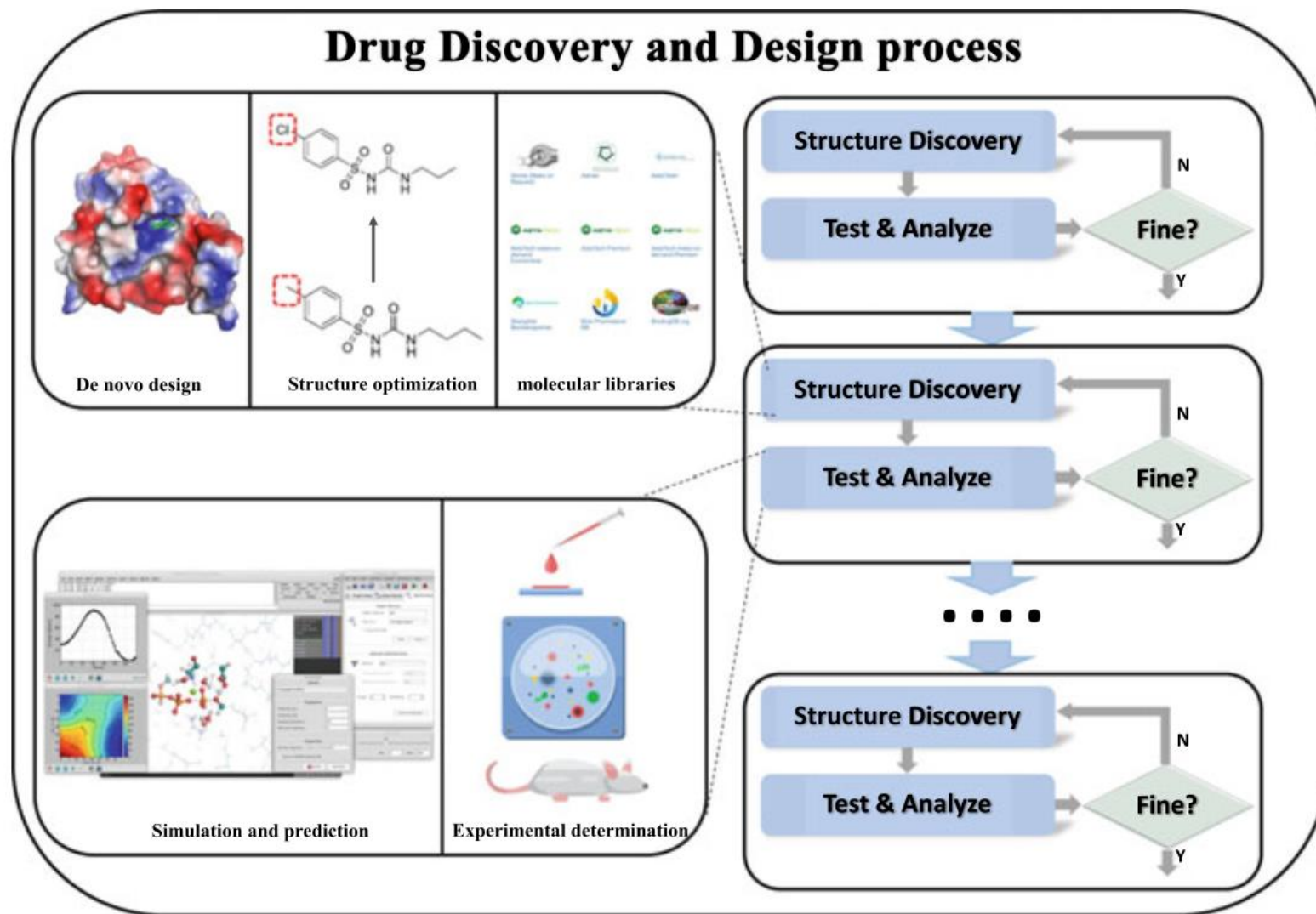


Figure from: Ziqiao Zhang et al. "GANs for molecular generation in drug design and discovery", Book chapter, 2022

Molecular representation

- **Molecular representation** indicates the form of molecules used as the input to neural networks for extracting the latent vectors of molecules
 - It acts as a carrier (or form) of molecules, and it is different from that in representation learning

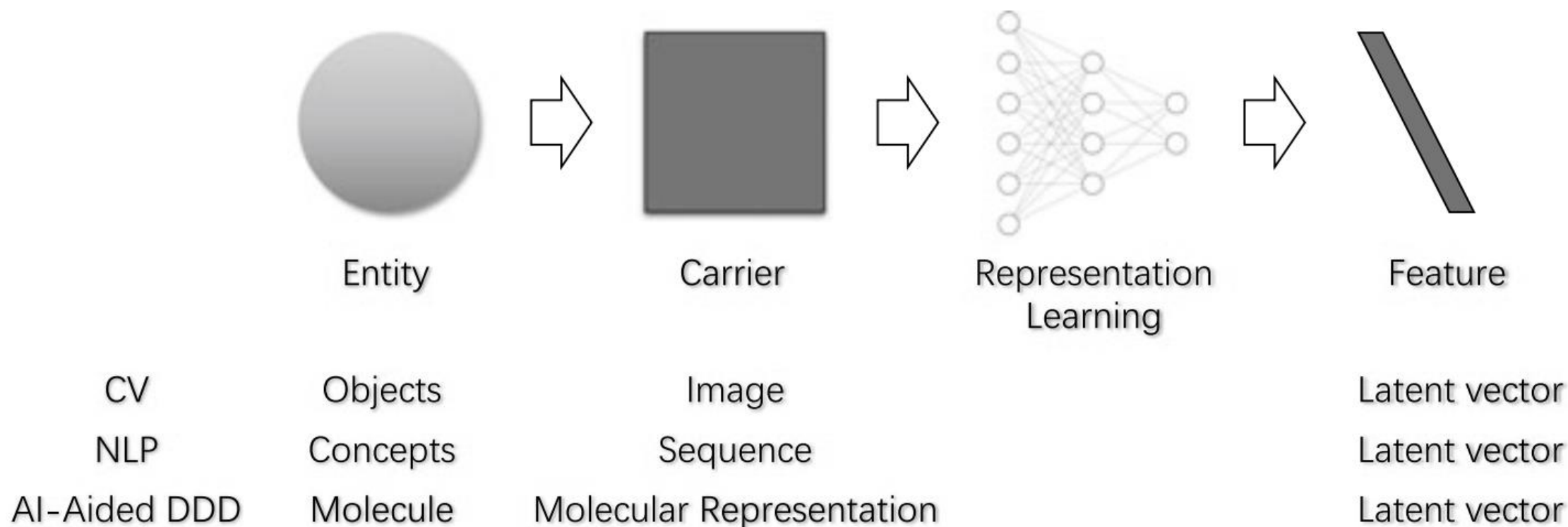
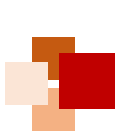


Figure from: Ziqiao Zhang et al. "GANs for molecular generation in drug design and discovery", Book chapter, 2022



Two types of molecular representation



- 2D molecular graphs
 - **SMILES**: encodes a molecular graph into a sequence of ASCII characters via depth-first graph traversal
 - **Fingerprint (FP)**: vectors describing the molecular structure with each bit describing the presence or absence of different molecular substructures
- 3D geometric structures
 - To capture 3D conformation information of a molecule, e.g. bond lengths, bond angles and dihedral angles
 - E.g. using 3D grid of voxels
 - It has not been popular, due to limitations, e.g. lack of uniqueness and invariance to transformations

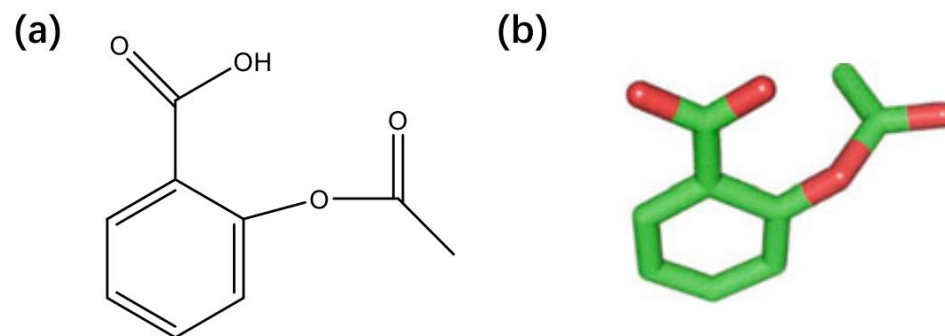


Fig. 11.3 **a** 2D molecular graph and **b** 3D geometric structure of an aspirin molecule. The SMILES representation of an aspirin is: OC(C1=C(OC(C)=O)C=CC=C1)=O



Evaluation: general statistical metrics



- **Validity:** To generate molecules that are valid in chemical sense
 - Valid molecules correspond to some realistic molecules, e.g. with correct SMILES syntax and valid valences
- **Uniqueness:** To generate different molecules
 - A model should explore the entire chemical space to generate a vast number of unique molecules
 - Rather than lazily generate duplicate molecules just to minimize the loss
- **Novelty:** To generate new molecules
 - A model should have a good coverage of the chemical space
 - To generate molecules similar to the ones in the training set, but not the same as them





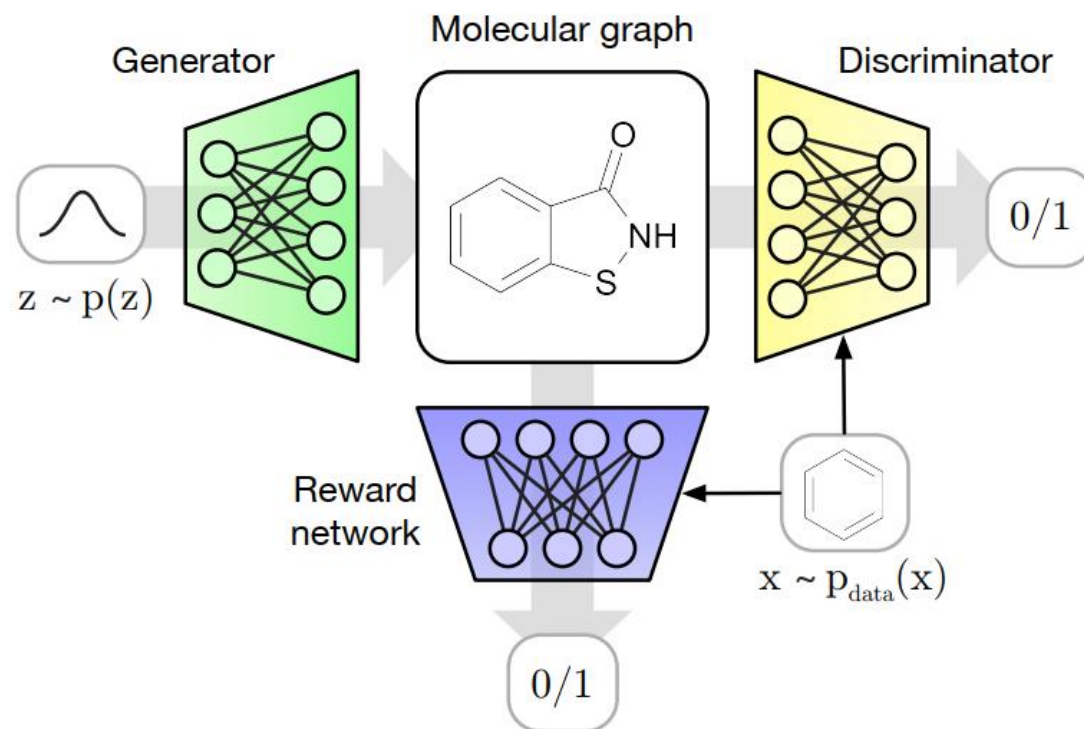
Evaluation: chemical statistical metrics



- **Bioactivity**: the binding affinity with a certain target protein
- **Drug-likeness**: how likely a molecule is to be a drug, e.g. molecular mass, hydrophobicity, polarity, etc.
- **Synthesizability**: how easily a molecule can be synthesized
- **Solubility**: whether a molecule is hydrophilic
- Specific molecular properties: molecular weight, the number of aromatic rings, TPSA, similarity to existing molecules, the presence or absence of some specific substructures, etc.
- The distance between the distribution of the chemical properties of the generate molecules and that in the training set
 - Fréchet ChemNet distance (FCD)
 - KL divergence



The MolGAN model

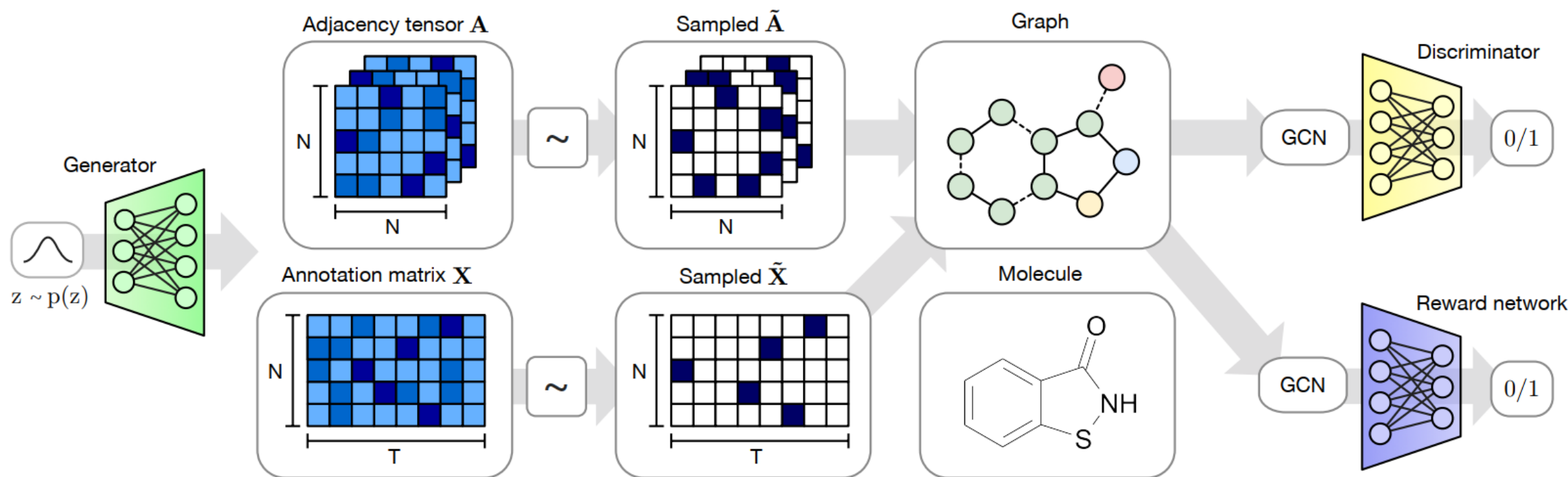


N.D. Cao and T. Kipf, ICML 2018

- MolGAN utilizes
 - **GAN**: to work directly on graph representations
 - **Reinforcement learning (RL)**: to encourage the generation of molecules with particular properties
- It consists 3 modules: generator, discriminator, and reward network



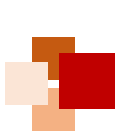
Outline of MolGAN



N.D. Cao and T. Kipf, ICML 2018

- From left to right:

- The generator takes a sample from a prior distribution and generates a dense adjacency tensor A and annotation matrix X
- Categorical sampling is used to obtain sparse and discrete matrices $\tilde{A}$ and $\tilde{X}$ from A and X
- Combination of $\tilde{A}$ and $\tilde{X}$ represents an annotated molecular graph corresponding to a specific chemical compound
- The graph is processed by the discriminator and reward networks, both based on Relational-GCN



Results



Objective	Algorithm	Valid (%)	Unique (%)	Time (h)	Diversity	Druglikeliness	Synthesizability	Solubility
Druglikeliness	ORGAN	88.2	69.4*	9.63*	0.55	0.52	0.32	0.35
	OR(W)GAN	85.0	8.2*	10.06*	0.95	0.60	0.54	0.47
	Naive RL	97.1	54.0*	9.39*	0.80	0.57	0.53	0.50
	<i>MolGAN</i>	99.9	2.0	1.66	0.95	0.61	0.68	0.52
	<i>MolGAN (QM9)</i>	100.0	2.2	4.12	0.97	0.62	0.59	0.53
Synthesizability	ORGAN	96.5	45.9*	8.66*	0.92	0.51	0.83	0.45
	OR(W)GAN	97.6	30.7*	9.60*	1.00	0.20	0.75	0.84
	Naive RL	97.7	13.6*	10.60*	0.96	0.52	0.83	0.46
	<i>MolGAN</i>	99.4	2.1	1.04	0.75	0.52	0.90	0.67
	<i>MolGAN (QM9)</i>	100.0	2.1	2.49	0.95	0.53	0.95	0.68
Solubility	ORGAN	94.7	54.3*	8.65*	0.76	0.50	0.63	0.55
	OR(W)GAN	94.1	20.8*	9.21*	0.90	0.42	0.66	0.54
	Naive RL	92.7	100.0*	10.51*	0.75	0.49	0.70	0.78
	<i>MolGAN</i>	99.8	2.3	0.58	0.97	0.45	0.42	0.86
	<i>MolGAN (QM9)</i>	99.8	2.0	1.62	0.99	0.44	0.22	0.89
All/Alternated	ORGAN	96.1	97.2*	10.2*	0.92	0.52	0.71	0.53
All/Simultaneously	<i>MolGAN</i>	97.4	2.4	2.12	0.91	0.47	0.84	0.65
All/Simultaneously	<i>MolGAN (QM9)</i>	98.0	2.3	5.83	0.93	0.51	0.82	0.69

N.D. Cao and T. Kipf, ICML 2018



Results (continued)

Algorithm	Valid	Unique	Novel
CharacterVAE	10.3	67.5	90.0
GrammarVAE	60.2	9.3	80.9
GraphVAE	55.7	76.0	61.6
GraphVAE/imp	56.2	42.0	75.8
GraphVAE NoGM	81.0	24.1	61.0
MolGAN	98.1	10.4	94.2

Table 3. Comparison with different algorithms on QM9. Values are reported in percentages. Baseline results are taken from [Simonovsky & Komodakis \(2018\)](#).

N.D. Cao and T. Kipf, ICML 2018

- Strengths of MolGAN:
 - Significantly higher validity scores than VAE-based baselines
 - It achieves both high validity and novelty scores
- Limitation of MolGAN: It is prone to **mode collapse**
 - Both GAN and RL objective do not encourage generation of diverse outputs whereby the model tends to be pulled towards a solution that only involves little sample variability
 - This ultimately results in the generation of only a handful of different molecules if training is not stopped early

